Documentazione Tecnica di Progetto

GiaLoSai

Gestione Logistica e Stoccaggio

Gruppo 11

Corso di Ingegneria del Software

Anno Accademico 2025–2026

25 Agosto 2026

`Versione v2.0.0`

Università di Bologna — Scuola di Ingegneria e Architettura

Storico delle Versioni

Versione	Data	Descrizione
v1.0.0	09/05/2026	Fondazione del progetto: repository, documentazione, UML e struttura m
v1.1.0	01/07/2026	Autenticazione JWT, persistenza MySQL, test di unità e Docker Compos
v1.2.0	27/07/2026	Client TUI con login, pagine magazzino e API lato client
v1.3.0	12/08/2026	Pagine complete (contabilità, admin), creazione ordini, DDT e gestione ut
v2.0.0	21/08/2026	Integrazione finale: allineamento client–server, bugfix, test stabili e docum

Contents

1 Introduzione **3**
1.1 Scopo del Progetto . . . 3
1.2 Obiettivi e Ambito (Scope) . . . 3
1.3 Glossario . . . 3

2 Analisi dei Requisiti **3**
2.1 Attori del Sistema . . . 3
2.2 Requisiti Funzionali (RF) . . . 3
2.3 Requisiti Non Funzionali (RNF) . . . 5
2.4 Analisi dei Casi d'Uso . . . 5

3 Progettazione e Architettura **6**
3.1 Scelte Architetturali . . . 6
3.2 Modello del Dominio . . . 7
3.3 Design Dettagliato (Diagramma delle Classi) . . . 7
3.4 Comportamento del Sistema (Diagrammi di Sequenza) . . . 8

4 Gestione del Progetto (Scrum) **8**
4.1 Backlog . . . 8
4.2 User Stories . . . 10
4.3 Criteri di Accettazione . . . 12

5 Strategia di Verifica e Validazione **12**
5.1 Piano di Testing . . . 12
5.2 Casi di Test (Test Cases) . . . 13

1 Introduzione

1.1 Scopo del Progetto

Il software ha l'obiettivo di gestire le dinamiche logistiche interne di un'azienda, ottimizzando la gestione del magazzino, della movimentazione delle merci in entrata/uscita e della documentazione contabile associata.

> **Nota:** Il progetto si ispira a soluzioni CLI già consolidate nel panorama dello sviluppo software: `docker` per la gestione containerizzata, `kubectl` per l'amministrazione di cluster, e `gh` (GitHub CLI) per l'interazione con piattaforme remote. GiaLoSai adotta lo stesso principio di interfaccia testuale affidabile e navigabile da tastiera.

1.2 Obiettivi e Ambito (Scope)

L'implementazione è un sistema client–server destinato a una rete locale: il client TUI comunica con il backend HTTP, che accede a MySQL. Il progetto modella l'intero ciclo della merce, dalla registrazione dei prodotti fino a DDT, fatture, note di credito e resi.

1.3 Glossario

- **DDT**: Documento di Trasporto.
- **RBAC**: Role-Based Access Control (Gestione permessi basata sui ruoli).
- **TUI**: Text-based User Interface, interfaccia utente da terminale.
- **DTO**: Data Transfer Object, struttura usata per lo scambio HTTP.
- **Repository**: componente che incapsula l'accesso alla persistenza.
- **Stock-out**: Condizione di esaurimento delle scorte di un prodotto.

2 Analisi dei Requisiti

2.1 Attori del Sistema

- **Magazziniere**: Gestisce il flusso fisico delle merci (carico/scarico), effettua la scansione dei codici a barre, consulta le giacenze e genera i DDT per la merce in uscita.
- **Contabile**: Visiona l'elenco e il dettaglio degli ordini, gestisce le fatture, le note di credito e modifica i listini dei prezzi di acquisto e vendita.
- **Admin**: Supervisione globale del sistema, gestione degli utenti, consultazione delle merci e dei prezzi, controllo delle fatture, gestione dei resi e generazione della reportistica avanzata dell'inventario.

2.2 Requisiti Funzionali (RF)

Table 1: Requisiti Funzionali

ID	Descrizione	Priorità
RF-01	Login degli utenti tramite username e password basato su ruoli (RBAC).	Alta
RF-02	Blocco automatico dell'account utente a seguito di 3 tentativi di login falliti (l'account admin è esente).	Alta
RF-03	Recupero delle credenziali tramite l'admin, che sblocca l'account e reimposta la password dell'utente.	Media
RF-04	Visualizzazione, inserimento e modifica dei dati anagrafici dei prodotti e delle categorie.	Alta
RF-05	Consultazione in tempo reale dell'elenco delle giacenze di magazzino.	Alta
RF-06	Registrazione di un nuovo Ordine di Entrata e aggiornamento automatico incrementale delle giacenze.	Alta
RF-07	Registrazione di un nuovo Ordine di Uscita e decremento automatico delle giacenze dei prodotti coinvolti.	Alta
RF-08	Generazione automatica di un DDT per ogni Ordine di Uscita evaso.	Alta
RF-09	Passaggio automatico dello stato dell'Ordine di Uscita da "Confermato" a "Evaso" alla creazione del DDT.	Alta
RF-10	Notifica visiva immediata per i prodotti la cui giacenza scende sotto la soglia minima di sicurezza.	Media
RF-11	Consultazione e gestione dei resi della merce con relativo ripristino delle quantità in giacenza.	Media
RF-12	Visualizzazione e filtraggio degli ordini ricevuti (evasi e non evasi) da parte del contabile.	Alta
RF-13	Generazione e conservazione di fatture di vendita e note di credito con numerazione progressiva annuale.	Alta
RF-14	Modifica manuale dei listini dei prezzi di vendita e di acquisto dei prodotti.	Media
RF-15	Integrazione software per l'acquisizione rapida dei codici a barre dei prodotti tramite lettore ottico.	Media

RF-16	Gestione degli utenti: creazione, modifica, eliminazione e visualizzazione degli account da parte dell'admin.	Alta

2.3 Requisiti Non Funzionali (RNF)

Table 2: Requisiti Non Funzionali

ID	Categoria	Descrizione
RNF-01	Prestazioni	Il tempo di risposta alla scansione di un codice a barre deve essere inferiore a 0.5s per il 95% delle operazioni.
RNF-02	Prestazioni	La generazione di un report annuale dell'inventario non deve richiedere più di 3s di elaborazione CPU.
RNF-03	Portabilità	Funzionamento garantito in modalità offline su qualsiasi sistema operativo desktop moderno.
RNF-04	Sicurezza	Dati archiviati interamente in locale senza alcuna esposizione o transito all'esterno della rete aziendale.
RNF-05	Archiviazione	Persistenza relazionale centralizzata in MySQL 8, avviabile con Docker Compose.
RNF-06	Usabilità	Interfaccia testuale (TUI) leggera, navigabile da terminale.

2.4 Analisi dei Casi d'Uso

Il sistema è documentato tramite diagrammi dei casi d'uso che mappano le interazioni tra i tre attori principali (magazziniere, contabile e admin) e le macro-funzionalità: ordini, documenti fiscali, resi, inventario, prodotti e prezzi, utenti e autenticazione. Le relazioni di inclusione (*include*) e di estensione (*extend*) modellano rispettivamente i passi obbligati e le varianti dei flussi. I flussi dettagliati delle operazioni principali sono descritti nei diagrammi di sequenza e di attività allegati nella cartella di progetto.

Nota: I diagrammi seguenti sono esportati dai sorgenti PlantUML in `documentazione/uml/` e descrivono il modello di dominio e i flussi di design allineati al codice. La disponibilità effettiva di ogni funzionalità resta determinata dalle rotte del router e dalla navigazione della TUI: i diagrammi sono una vista di design, non una garanzia di integrazione end-to-end.

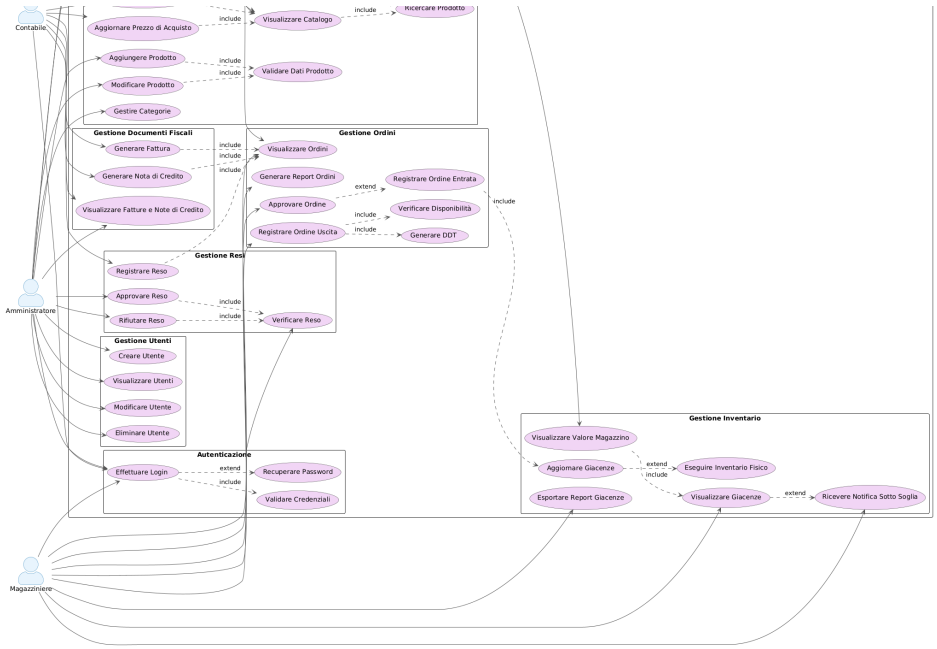


Figure 1: Diagramma dei Casi d'Uso Generale

3 Progettazione e Architettura

3.1 Scelte Architetturali

Il sistema combina gli stili *client–server*, *layered* e *repository*. Il client TUI è separato dal backend REST e comunica mediante HTTP/JSON. Nel server il router e il middleware costituiscono l'adattatore di trasporto; controller, service e repository separano rispettivamente conversione dei DTO, regole applicative e persistenza. Non si tratta di MVC classico: la View appartiene al processo client, mentre i controller del server sono controller HTTP.

> **Nota:** L'architettura client–server con TUI segue lo stesso pattern utilizzato da strumenti come `gh` (GitHub CLI) e `docker`: un client leggero che espone un'interfaccia testuale ricca, comunicando con un backend tramite API REST. Il pattern Repository è lo stesso adottato da `database/sql` in Go e da ORM come GORM, garantendo separazione tra logica applicativa e persistenza.

I repository incapsulano query MySQL parametrizzate. I service dipendono da interfacce ristrette e sono quindi verificabili con mock, coerentemente con Dependency Inversion. `TransactionHelper` propaga la transazione nel contesto, garantendo atomicità per operazioni quali aggiornamento delle giacenze e registrazione degli ordini. MySQL 8, InnoDB e le chiavi esterne sono configurati nello script `docker/mysql/init.sql`.

3.2 Modello del Dominio

Il dominio dell'applicazione è incentrato sulle entità fondamentali per il tracciamento logistico, mappate nel file `Domain_model.puml`: `Utente` e `Sessione`, `Categoria`, `Prodotto` e `Giacenza`, `OrdineEntrata` e `OrdineUscita` con le rispettive righe, `DDT`, `Fattura` (comprese le note di credito, distinte dal tipo) e `Reso`.

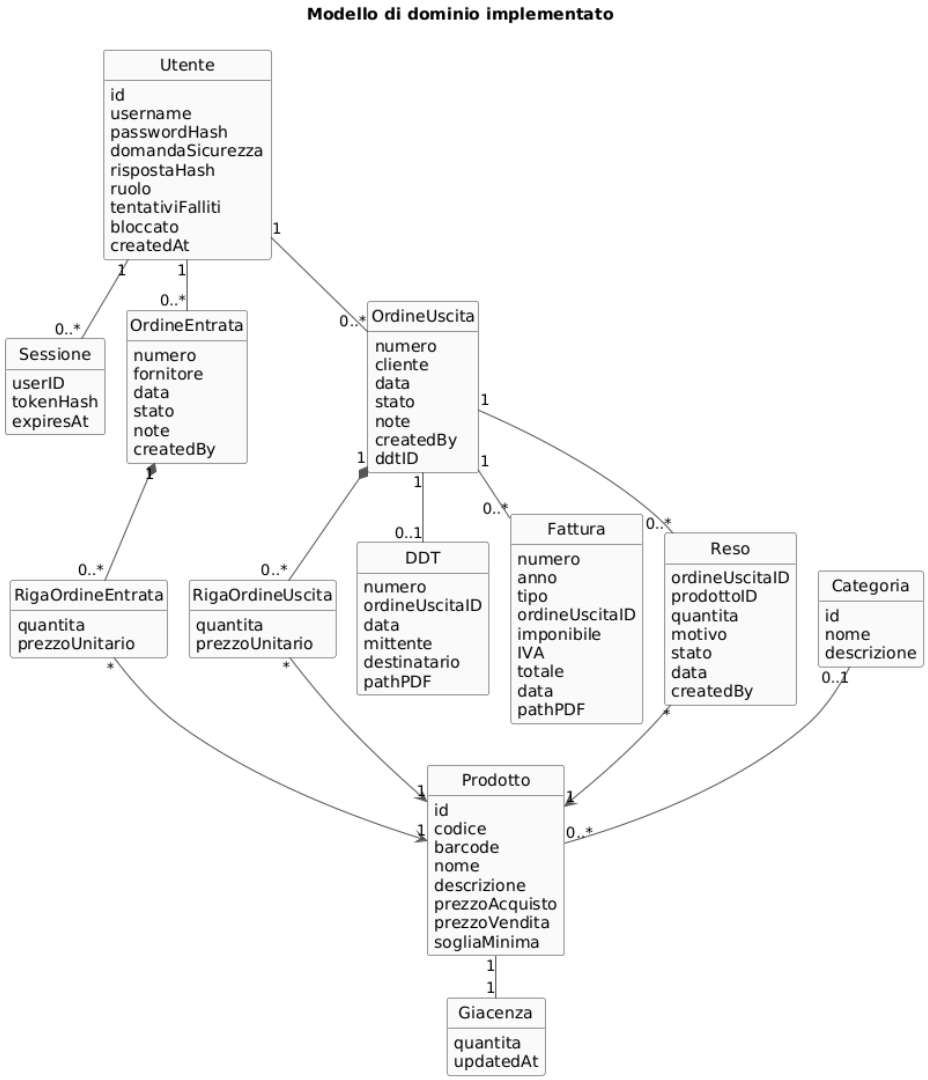


Figure 2: Modello del Dominio

3.3 Design Dettagliato (Diagramma delle Classi)

Il modello del codice comprende entità di dominio, DTO, controller, service e repository. Il pattern Repository isola il dialetto SQL e il dettaglio della connessione dal service layer. L'aggregazione dei componenti avviene in `router.NewRouter`; il dominio rimane indipendente da HTTP e da MySQL. Il diagramma delle classi è una vista di design e deve essere letto insieme al codice: le operazioni effettivamente pubblicate sono determinate dalle rotte del router.

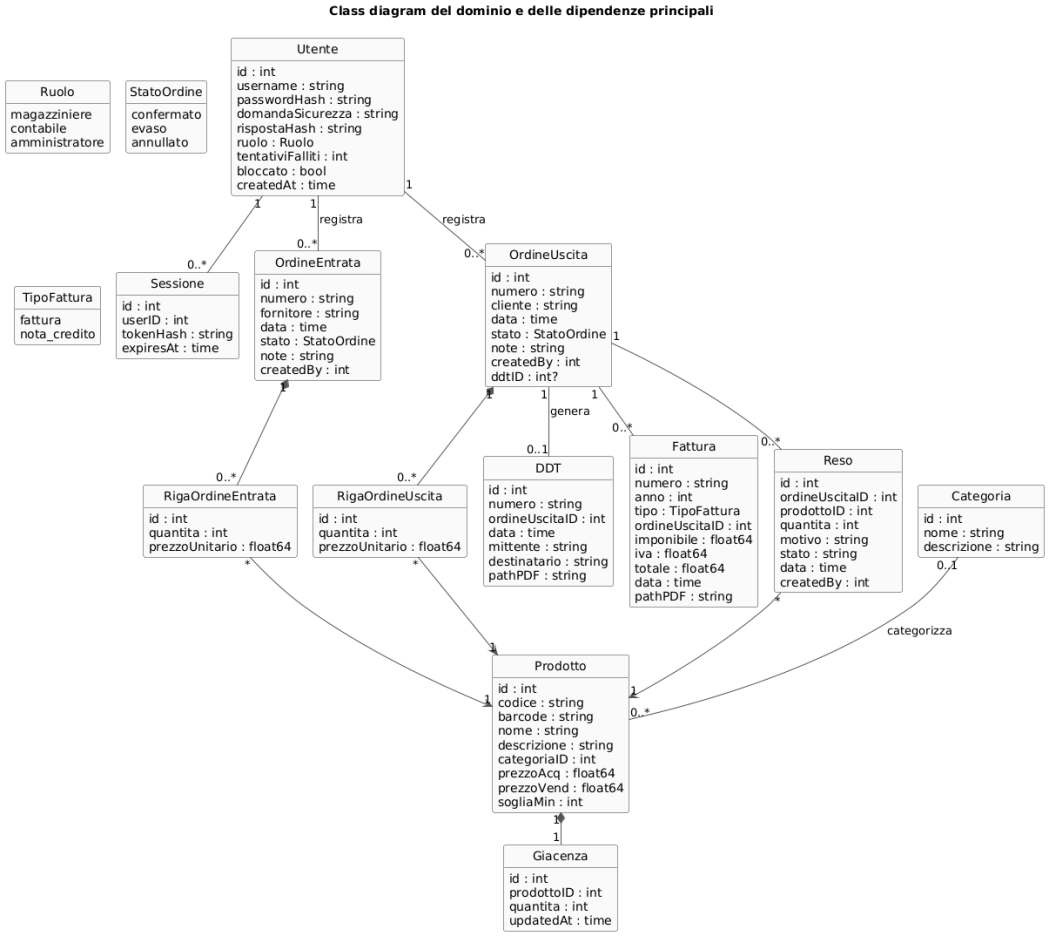


Figure 3: Diagramma delle Classi

3.4 Comportamento del Sistema (Diagrammi di Sequenza)

I diagrammi rappresentano scenari di requisiti e design. I flussi implementati devono essere distinti da quelli pianificati: login, consultazione di prodotti e categorie, aggiornamento del prezzo di vendita e lettura dei documenti pubblicati dal router sono esposti in HTTP. DDT, fatture, resi e ordini sono presenti nei service, ma non tutte le relative azioni hanno una rotta o una navigazione TUI disponibile. I diagrammi non costituiscono quindi una garanzia di disponibilità end-to-end.

4 Gestione del Progetto (Scrum)

4.1 Backlog

L'elenco ordinato e prioritizzato dei requisiti tecnici necessari al completamento del primo ciclo di sviluppo comprende:

1. Configurazione dell'architettura software di base, dell'ambiente Docker e della connessione a MySQL.

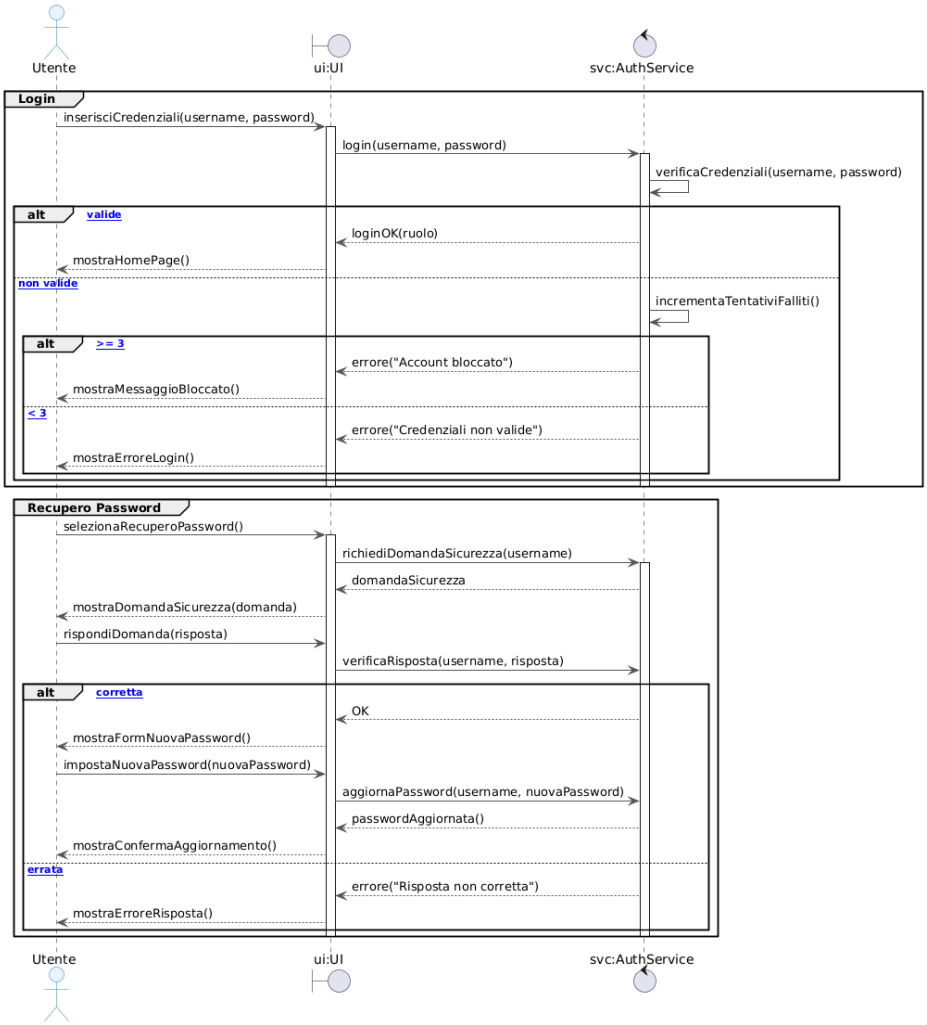


Figure 4: Diagramma di Sequenza: Login e Recupero Password (UC1)

2. Definizione dello schema logico della base dati tramite script SQL (creazione delle tabelle `Utente`, `Prodotto`, `Categoria`, `Giacenza`, `OrdineEntrata`, `OrdineUscita`, `RigaOrdineEntrata`, `RigaOrdineUscita`, `DDT`).
3. Sviluppo delle interfacce grafiche e delle funzioni CRUD per la gestione dell'Anagrafica Prodotti e Categorie.
4. Implementazione del modulo di movimentazione (logica e maschere per la compilazione di `OrdineEntrata` e `OrdineUscita`).
5. Integrazione software per l'acquisizione dell'input da lettore di codici a barre tramite emulazione tastiera.
6. Sviluppo del sottosistema di controllo e notifica visiva per i prodotti sotto la soglia minima inseriti in `Giacenza`.
7. Sviluppo del motore di reportistica dell'inventario: il report JSON è esposto dal router; l'esportazione in formato PDF e CSV è pianificata ma non ancora implementata.
8. Sviluppo del modulo di autenticazione locale e gestione dei permessi basata sui ruoli tramite `AuthService`.

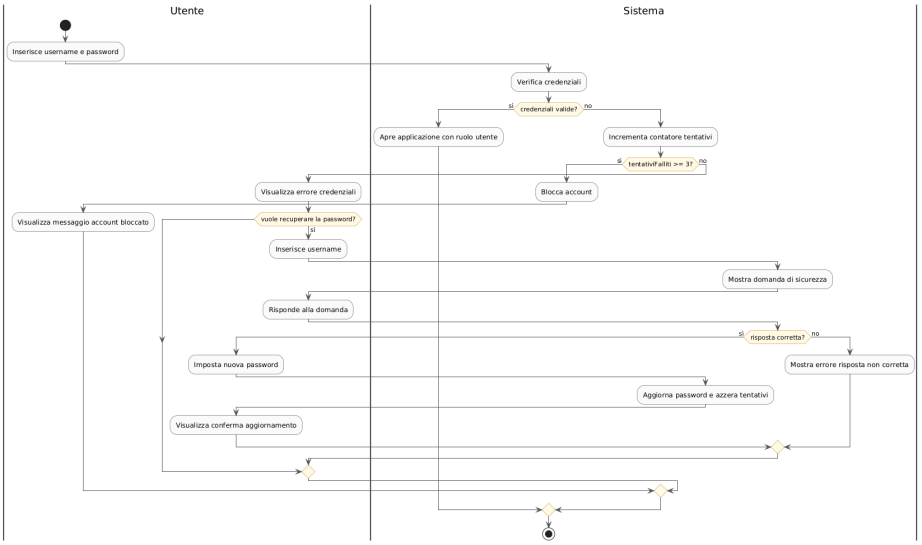


Figure 5: Diagramma di Attività: Login e Recupero Password (UC1)

4.2 User Stories

Le storie riflettono i bisogni concreti degli attori scritti secondo gli standard del framework Scrum:

- **US-01 (Accesso con credenziali)**: "Come utente, voglio accedere all'applicazione con username e password, in modo da poter utilizzare le funzionalità del mio ruolo".
- **US-02 (Scansione Barcode)**: "Come magazziniere, voglio poter scansionare il codice a barre di un prodotto in arrivo tramite il lettore ottico, per aggiornare la quantità in giacenza nel sistema in modo rapido, automatizzato e senza commettere errori manuali".
- **US-03 (Sicurezza Account)**: "Come admin del sistema, voglio che il software blocchi l'account dopo 3 tentativi falliti, per impedire ad utenti non autorizzati di indovinare le password d'accesso dei dipendenti".
- **US-04 (Recupero Credenziali)**: "Come utente bloccato o con credenziali dimenticate, voglio poter rivolgermi all'admin, che sblocca il mio account e mi fornisce una nuova password".
- **US-05 (Visualizzazione elenco ordini)**: "Come magazziniere, voglio vedere l'elenco di tutti gli ordini di entrata e uscita, in modo da avere una panoramica completa dei movimenti del magazzino".
- **US-06 (Visualizzazione dettaglio ordini)**: "Come magazziniere, voglio vedere il dettaglio di un ordine di entrata o uscita selezionato, in modo da conoscere i prodotti coinvolti e le relative quantità".
- **US-07 (Registrazione Ordini in entrata)**: "Come magazziniere, voglio registrare un ordine di entrata con i prodotti ricevuti, in modo da aggiornare automaticamente le giacenze".
- **US-08 (Registrazione Ordini in uscita)**: "Come magazziniere, voglio registrare un ordine di uscita con i prodotti richiesti, in modo da aggiornare automaticamente le giacenze".

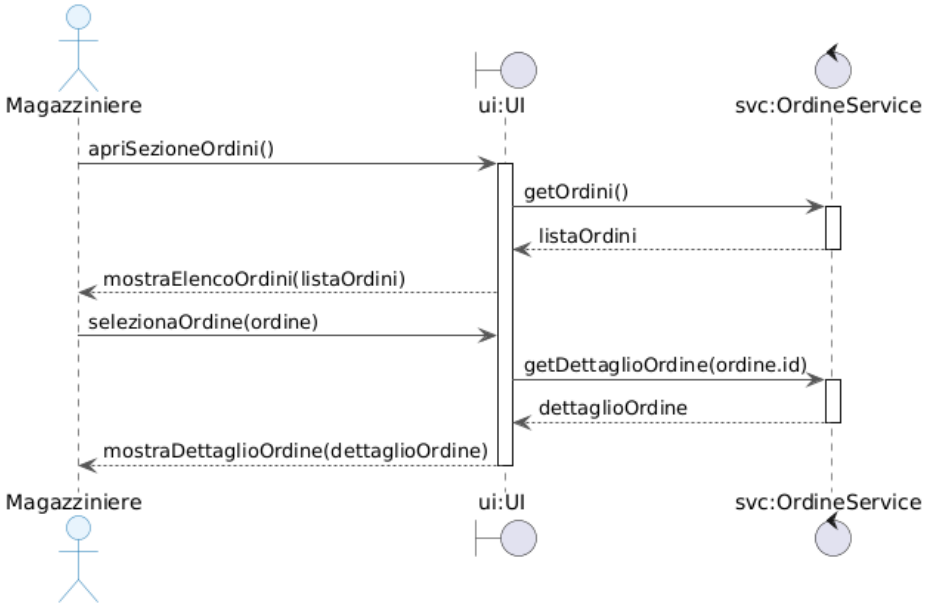


Figure 6: Diagramma di Sequenza: Visualizzazione Ordini (UC2)

- **US-09 (Generazione DDT)**: "Come magazziniere, voglio generare un documento di trasporto (DDT) per ogni ordine di uscita evaso, in modo da avere una documentazione ufficiale per la merce in uscita".
- **US-10 (Visualizzare Giacenze)**: "Come magazziniere, voglio visualizzare in tempo reale le giacenze di magazzino, in modo da sapere sempre la quantità disponibile di ogni prodotto".
- **US-11 (Stampa inventario)**: "Come magazziniere, voglio stampare un documento inventario, in modo da avere un registro fisico dello stato del magazzino".
- **US-12 (Gestione resi)**: "Come magazziniere, voglio gestire i resi della merce, in modo da poter ripristinare le quantità in giacenza e tenere traccia dei movimenti di reso".
- **US-13 (Visualizzazione ordini per contabile)**: "Come contabile, voglio visualizzare l'elenco degli ordini ricevuti (evasi e non evasi), in modo da poter gestire la fatturazione e le note di credito".
- **US-14 (Generazione Fatture)**: "Come contabile, voglio generare fatture per gli ordini di uscita evasi, in modo da poter emettere documenti ufficiali per i clienti".
- **US-15 (Nota di credito)**: "Come contabile, voglio generare una nota di credito per un ordine di uscita evaso, in modo da correggere la fatturazione quando necessario".
- **US-16 (Gestione prezzi)**: "Come contabile, voglio poter modificare manualmente i listini dei prezzi di vendita e di acquisto dei prodotti, in modo da mantenere aggiornati i prezzi nel sistema".
- **US-17 (Gestione utenti)**: "Come admin, voglio creare, modificare ed eliminare gli account degli utenti, in modo da gestire l'accesso al sistema e i relativi ruoli".
- **US-18 (Reportistica inventario)**: "Come admin, voglio generare report avanzati dell'inventario, in modo da avere una visione dettagliata dello stato del magazzino e delle tendenze di consumo dei prodotti".

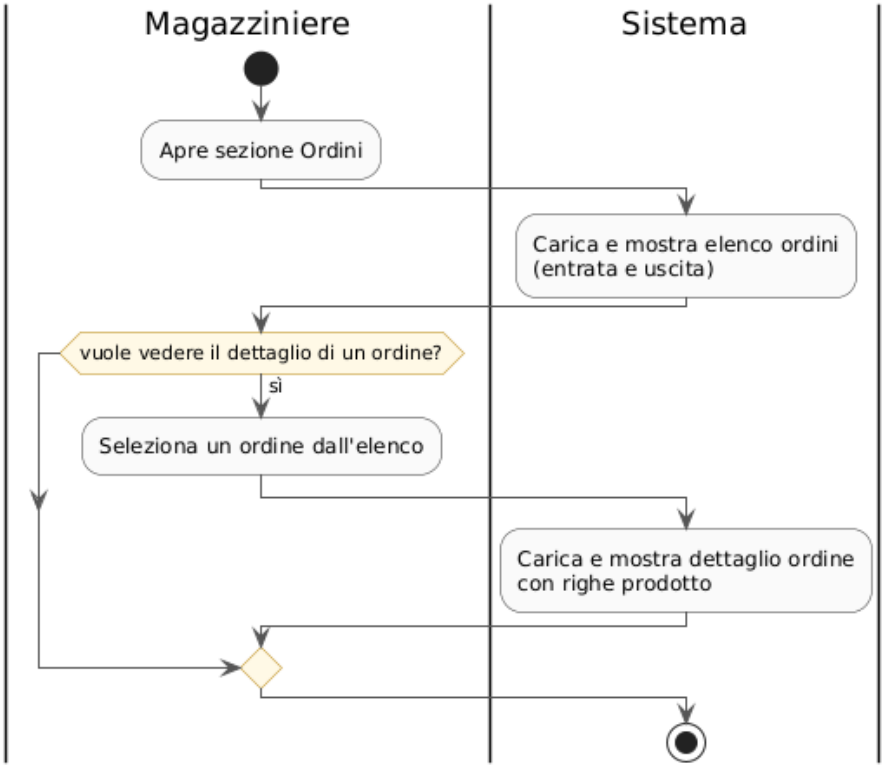


Figure 7: Diagramma di Attività: Visualizzazione Ordini (UC2)

4.3 Criteri di Accettazione

Vengono definiti i vincoli formali affinché la User Story **US-02** (Scansione Barcode) possa essere considerata completata ("Done"):

- **Criterio 1**: Il sistema deve riconoscere e decodificare correttamente e senza latenza i codici a barre conformi allo standard internazionale EAN-13.
- **Criterio 2**: Al momento della scansione riuscita all'interno della schermata di carico, i dati identificativi del prodotto devono comparire istantaneamente nei campi visivi e la relativa quantità nella tabella `Giacenza` deve essere incrementata automaticamente nel database.
- **Criterio 3**: Qualora il codice a barre non trovi corrispondenza in `Prodotti`, il service restituisce un errore di prodotto non trovato. Feedback acustico, pop-up e inserimento contestuale non sono implementati nella TUI corrente.

5 Strategia di Verifica e Validazione

5.1 Piano di Testing

Il piano prevede una suite di test di unità (Unit Testing) sistematici eseguiti sulla Business Logic tramite lo strumento standard di testing di Go (`go test`). I test sono focalizzati sulla verifica delle transazioni e sul rispetto dei vincoli di integrità del

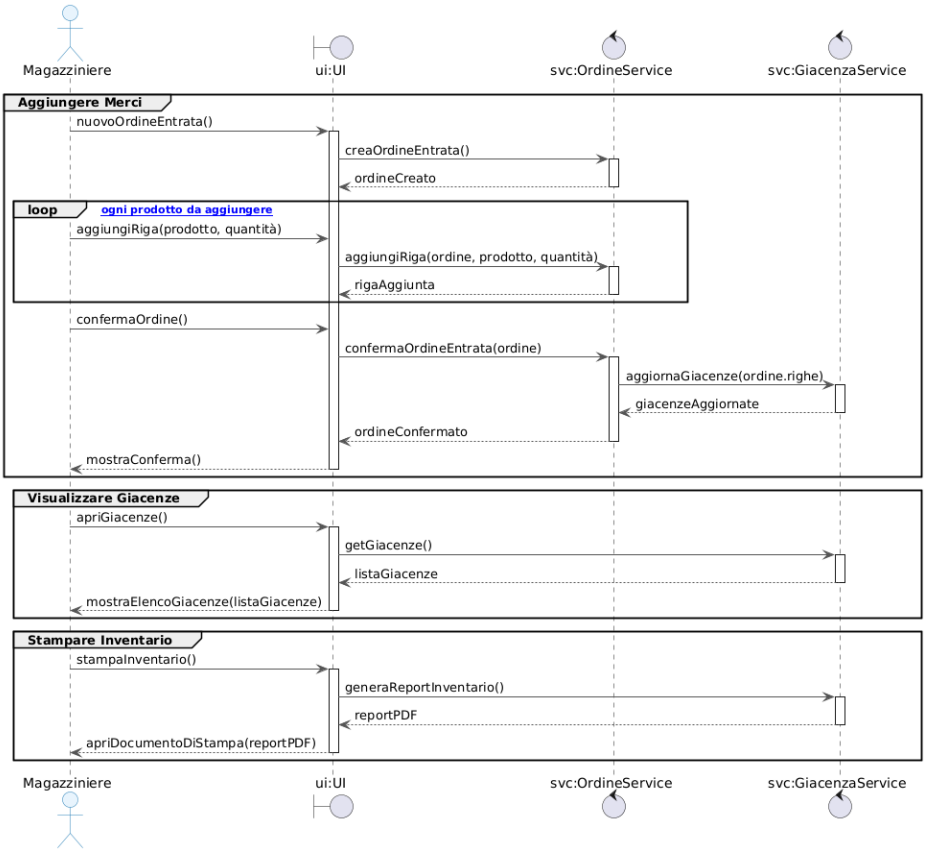


Figure 8: Diagramma di Sequenza: Gestione Giacenza (UC3)

magazzino.

> **Nota:** L'approccio ai test segue le best practice di ecosistemi CLI consolidati: `go test` è lo strumento standard anche per progetti come `docker`, `kubernetes` e `helm`. I test di integrazione con database transazionali sono un pattern comune nella struttura dei test di Go.

5.2 Casi di Test (Test Cases)

TC-01: Tentativo di scarico merce con quantità richiesta superiore alla giacenza fisica attuale.

- **Scenario**: Un prodotto registrato in anagrafica dispone di una quantità fisica in `Giacenza` pari a 5 unità. Viene inserito un `OrdineUscita` con una `RigaOrdineUscita` richiedente l'uscita di 6 unità del medesimo prodotto.
- **Input**: Invocazione della funzione di business `registraScarico()` con quantità = 6.
- **Risultato Atteso**: Il service intercetta l'incoerenza prima di modificare i dati, la transazione MySQL esegue il `ROLLBACK` e viene restituito un `GiacenzaInsufficienteError`. Il service impedisce quindi giacenze negative; la visualizzazione dell'errore dipende dall'integrazione TUI/API della specifica operazione.

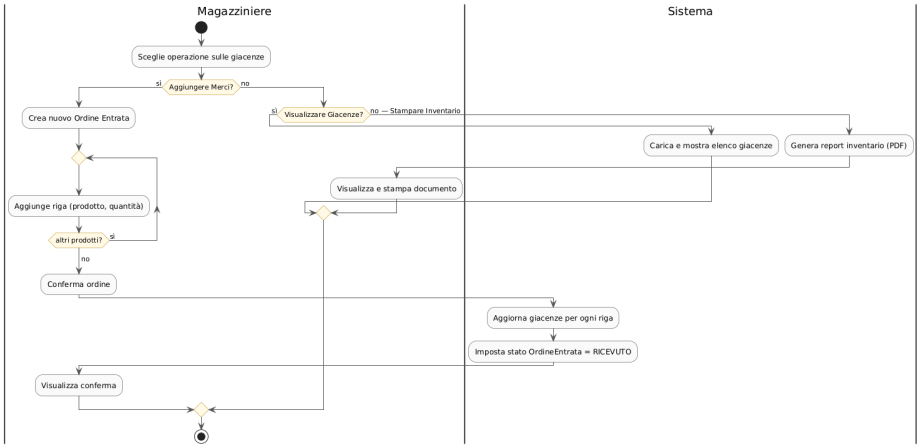


Figure 9: Diagramma di Attività: Gestione Giacenza (UC3)

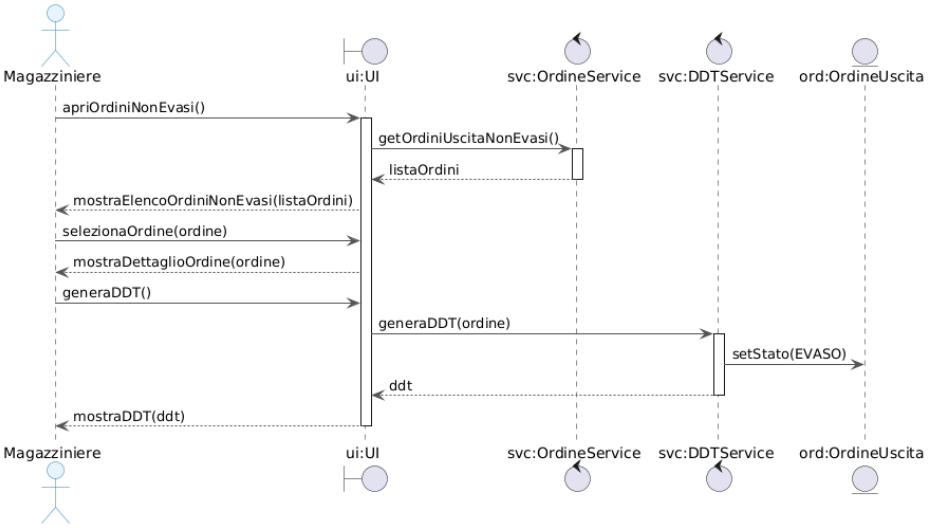


Figure 10: Diagramma di Sequenza: Generazione DDT (UC4)



Figure 11: Diagramma di Attività: Generazione DDT (UC4)

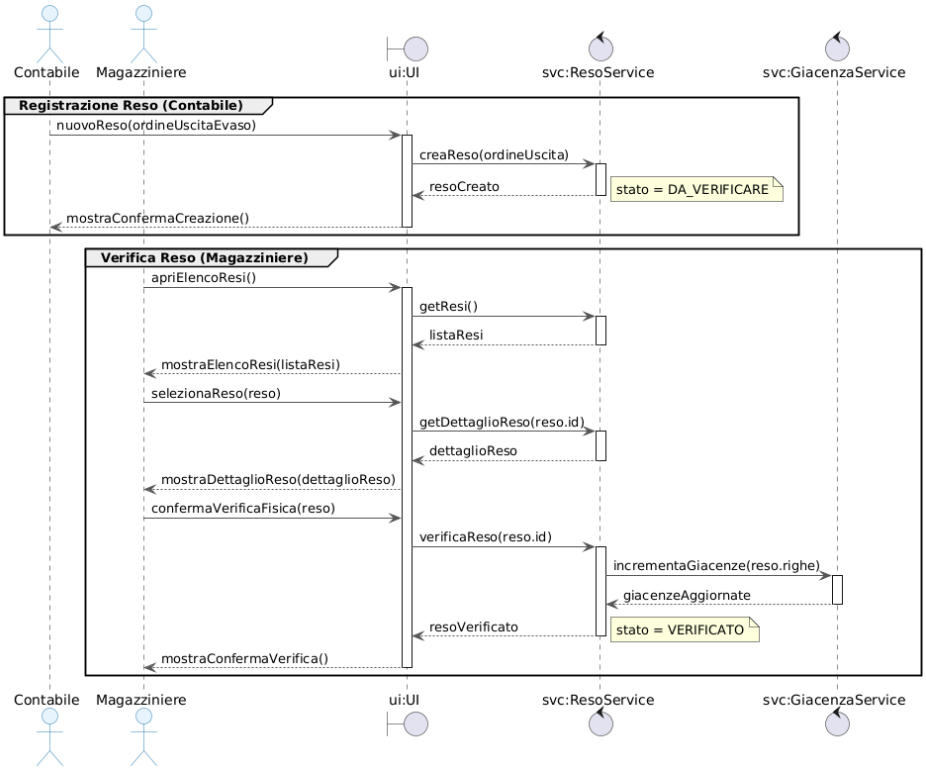


Figure 12: Diagramma di Sequenza: Resi (UC5)

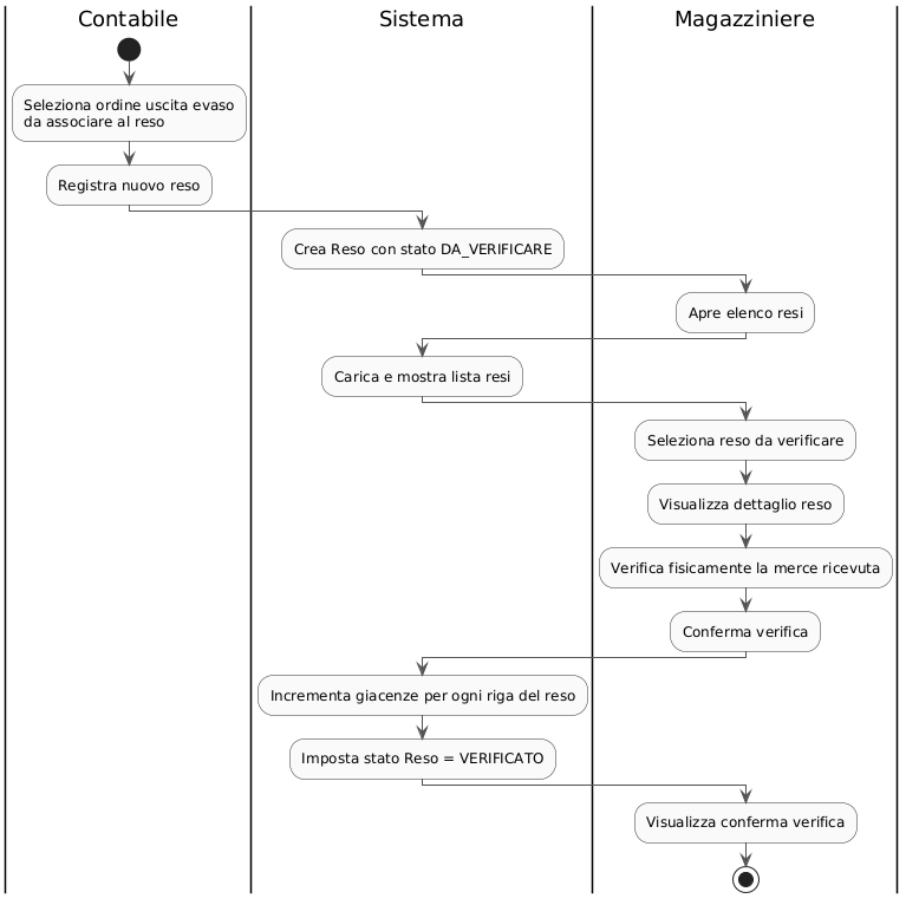


Figure 13: Diagramma di Attività: Resi (UC5)

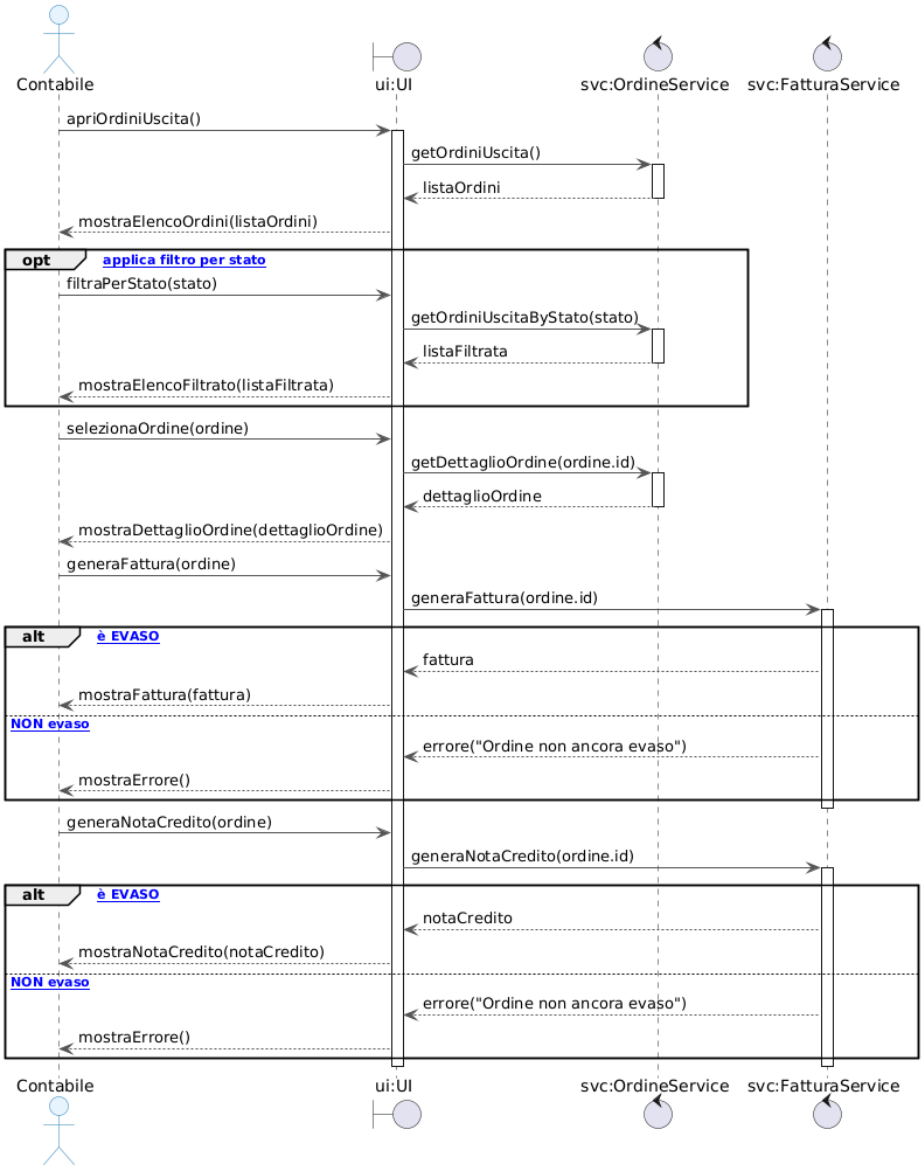


Figure 14: Diagramma di Sequenza: Contabile Ordini (UC6)

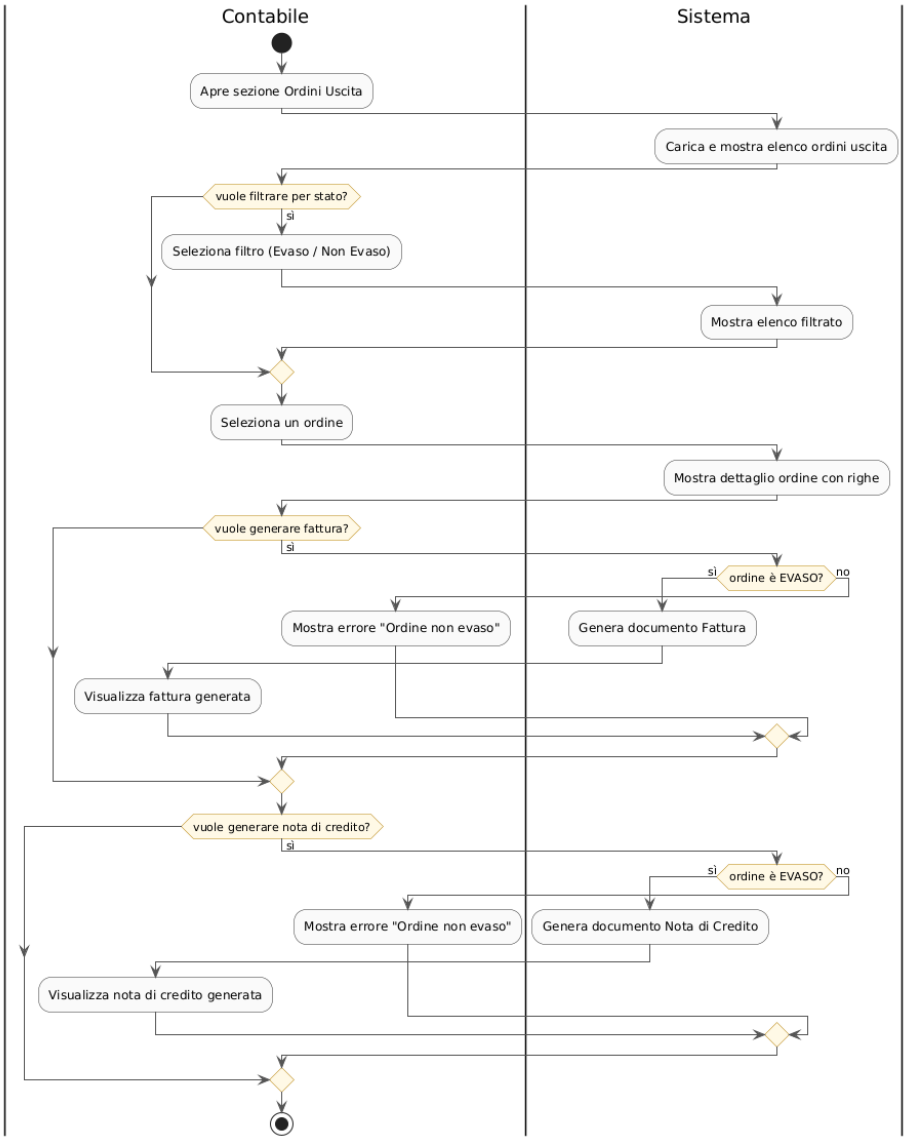


Figure 15: Diagramma di Attività: Contabile Ordini (UC6)

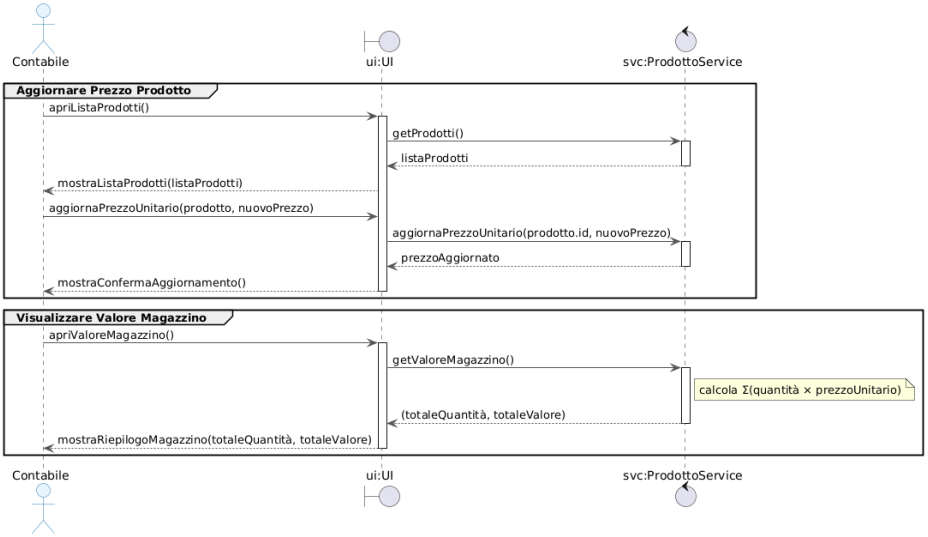


Figure 16: Diagramma di Sequenza: Contabile Prezzi (UC7)

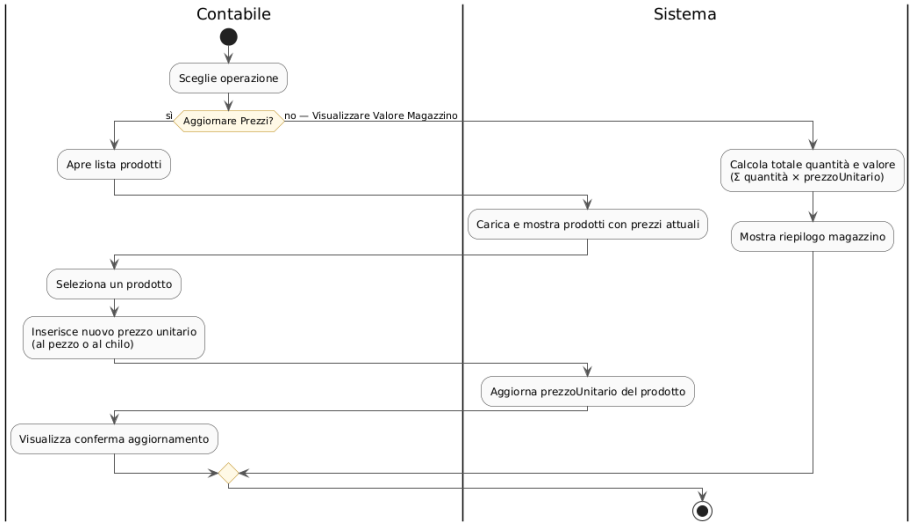


Figure 17: Diagramma di Attività: Contabile Prezzi (UC7)

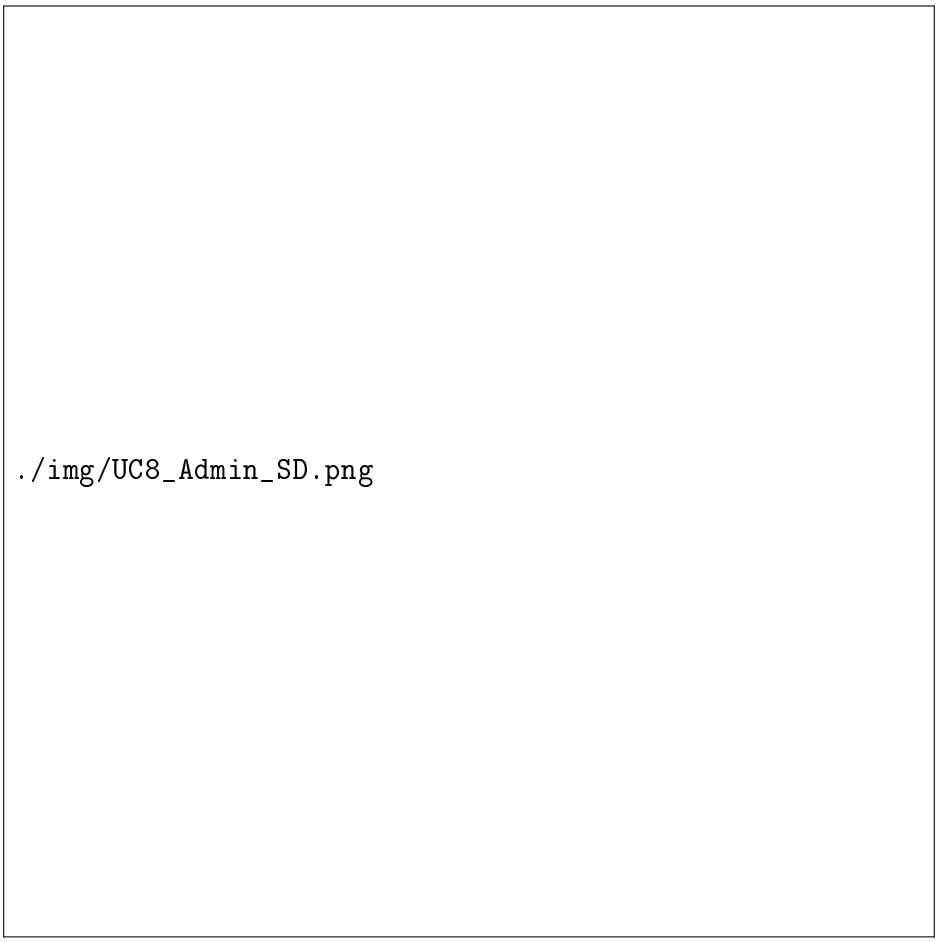


Figure 18: Diagramma di Sequenza: Admin (UC8)

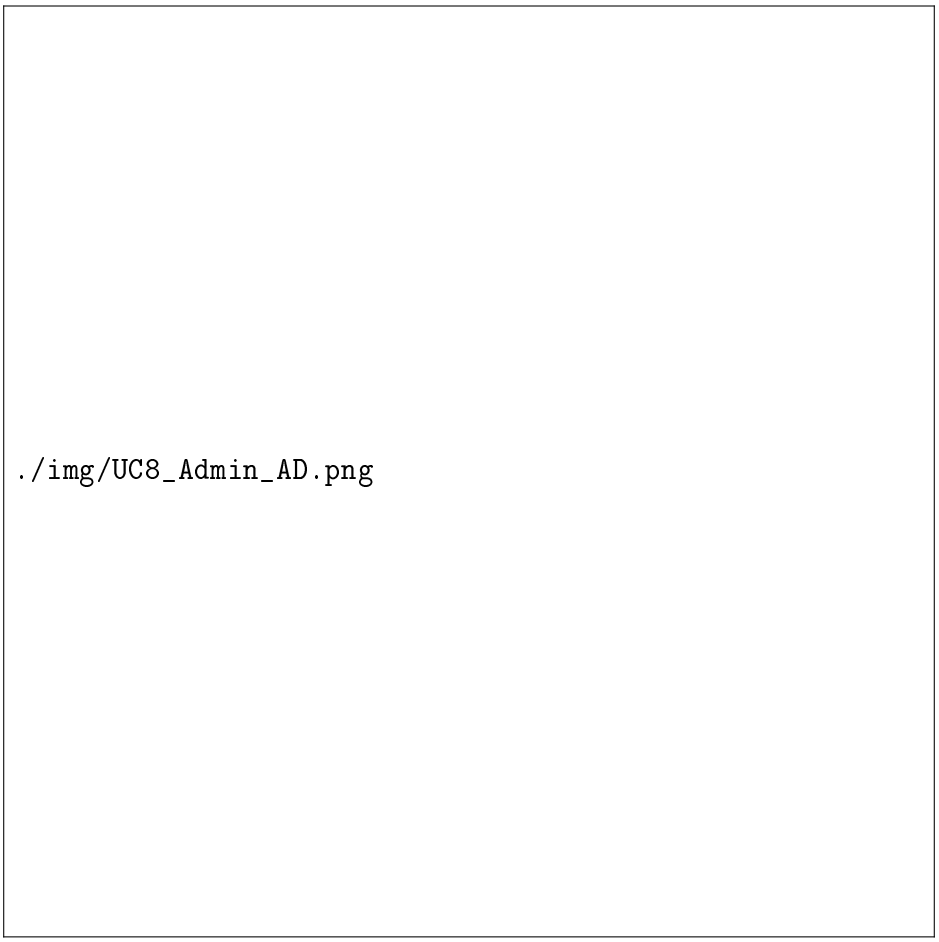


Figure 19: Diagramma di Attività: Admin (UC8)

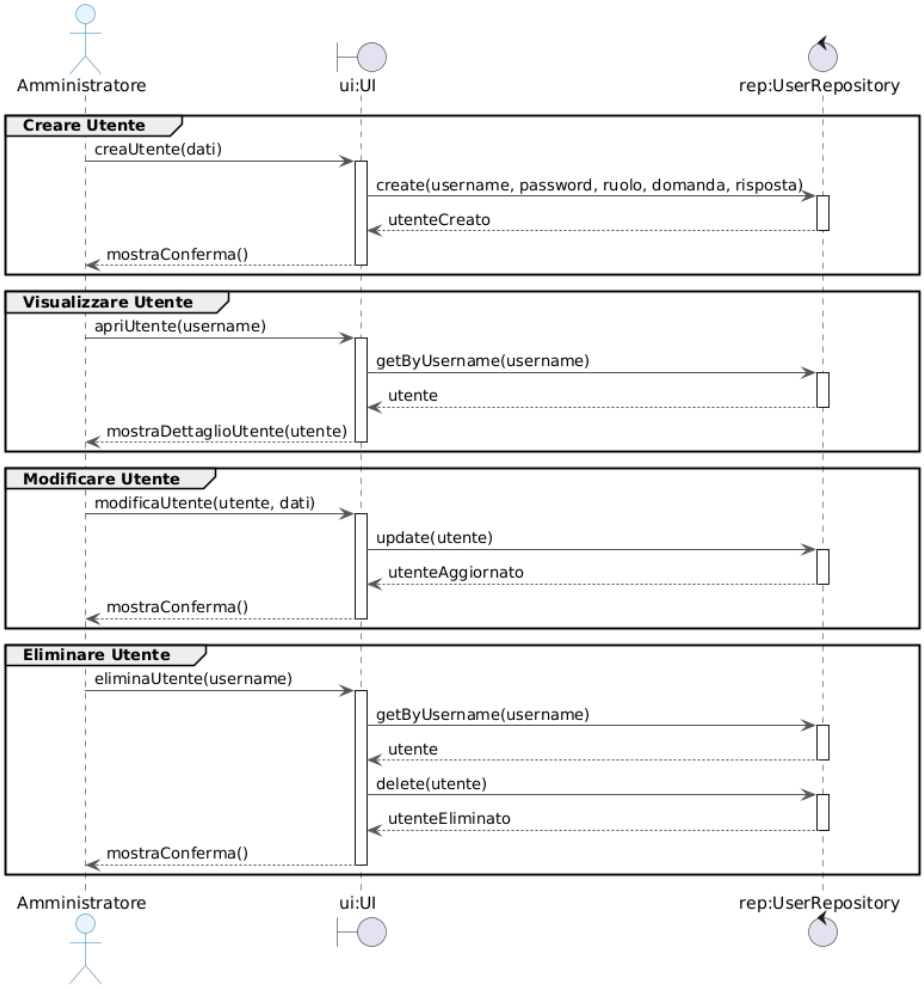


Figure 20: Diagramma di Sequenza: Gestione Utenti (UC9)

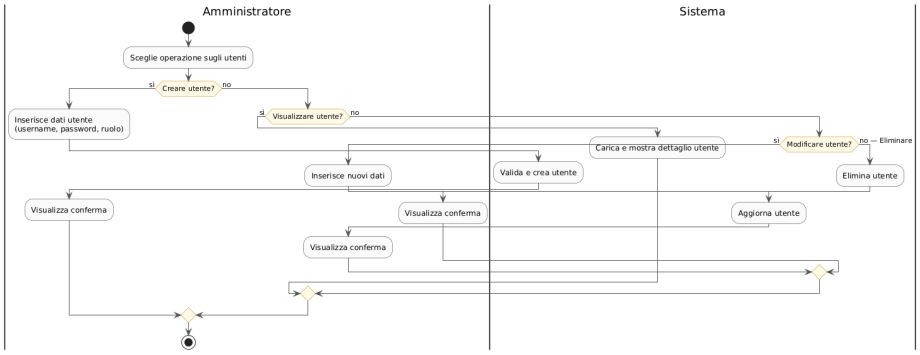


Figure 21: Diagramma di Attività: Gestione Utenti (UC9)